

Calcolatori Elettronici T
Ing. Informatica

Traccia soluzione
18 Luglio 2019

Esercizio 1

Nell'esercizio è necessario eseguire dei convenzionali trasferimenti dalle porte, una input e una output, ed evidenziare, mediante una richiesta di interruzione, quando sono stati eseguiti, su entrambe le porte, un numero dispari di trasferimenti.

Dispositivi e segnali presenti nel sistema.

Dispositivi di memoria:

RAM	mappata da E0000000h:FFFFFFFFh, 4 banchi da 128 MB
EPROM	mappata da 00000000h:3FFFFFFFh, 4 banchi da 256 MB

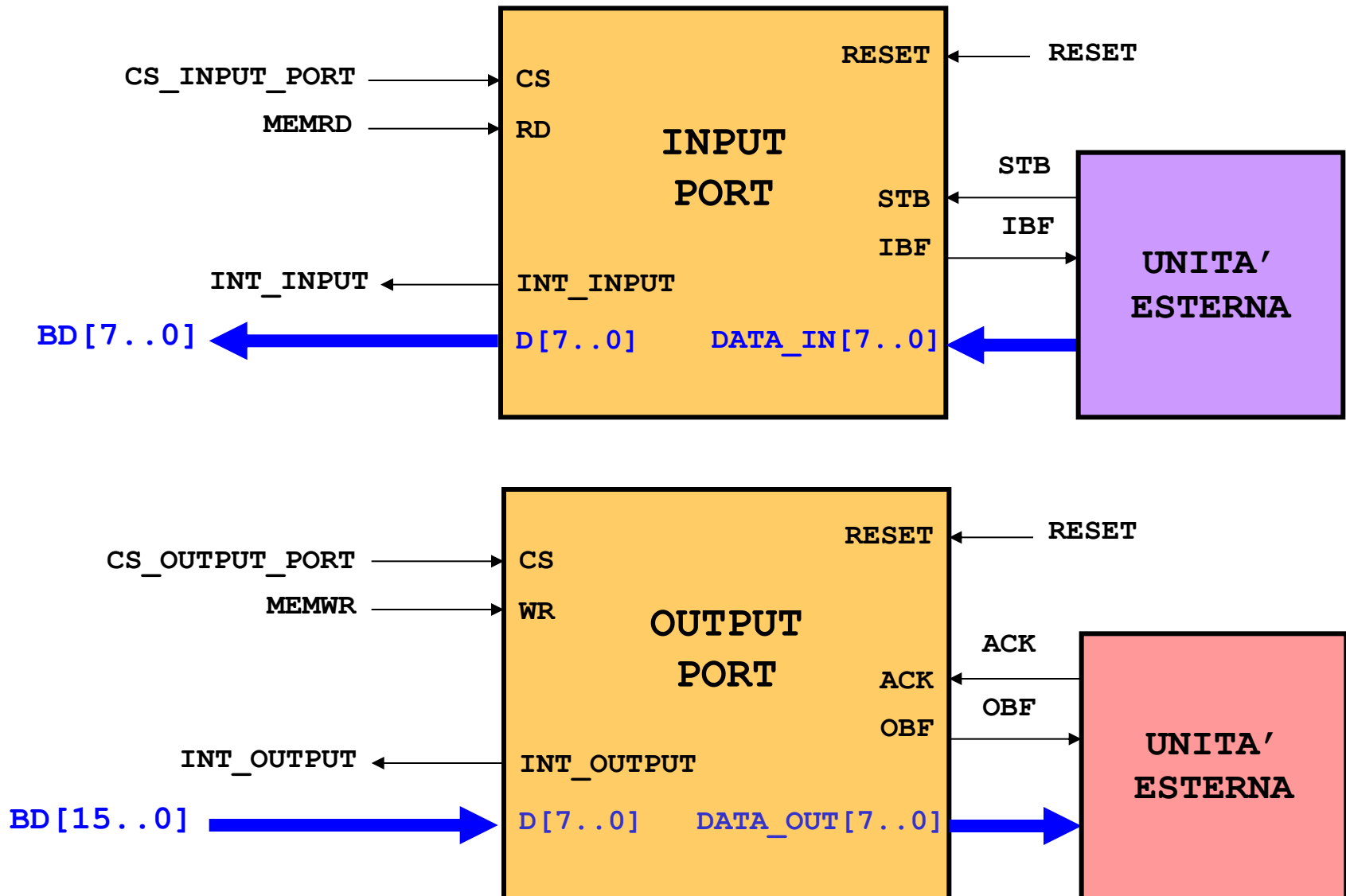
Porte di input, output e altri chip-select e/o segnali:

CS_INPUT_PORT	mappata a 80000000h
CS_OUTPUT_PORT	mappata a 80000001h
CS_READ_INT_STATUS	mappata a 80000002h
CS_INVERTI_LED	mappata a 80000003h

Segnali di decodifica di memorie, periferiche e segnali:

CS_RAM_0	= BA31 · BA30 · BE0	
CS_RAM_1	= BA31 · BA30 · BE1	
CS_RAM_2	= BA31 · BA30 · BE2	
CS_RAM_3	= BA31 · BA30 · BE3	
CS_INPUT_PORT	= BA31 · BA30* · BE0	mappato a 80000000h
CS_OUTPUT_PORT	= BA31 · BA30* · BE1	mappato a 80000001h
CS_READ_INT_STATUS	= BA31 · BA30* · MEMRD · BE2	mappato a 80000002h
CS_INVERTI_LED	= BA31 · BA30* · BE3	mappato a 80000003h
CS_EEPROM_0	= BA31* · BE0	
CS_EEPROM_1	= BA31* · BE1	
CS_EEPROM_2	= BA31* · BE2	
CS_EEPROM_3	= BA31* · BE3	

Nel sistema è presente una porta in input, INPUT_PORT, e una porta in output, OUTPUT_PORT, mappate rispettivamente su sui bus dati **BD[7..0]** e **BD[15..8]**.



Nel sistema sono presenti tre sorgenti di interruzione: quella generata da INPUT_PORT, quella generata da OUTPUT_PORT e quella generata dalla rete che monitora il numero dei trasferimenti. Pertanto, è necessario poter verificare, mediante una lettura a **CS_READ_INT_STATUS**, lo stato delle richieste di interruzione.

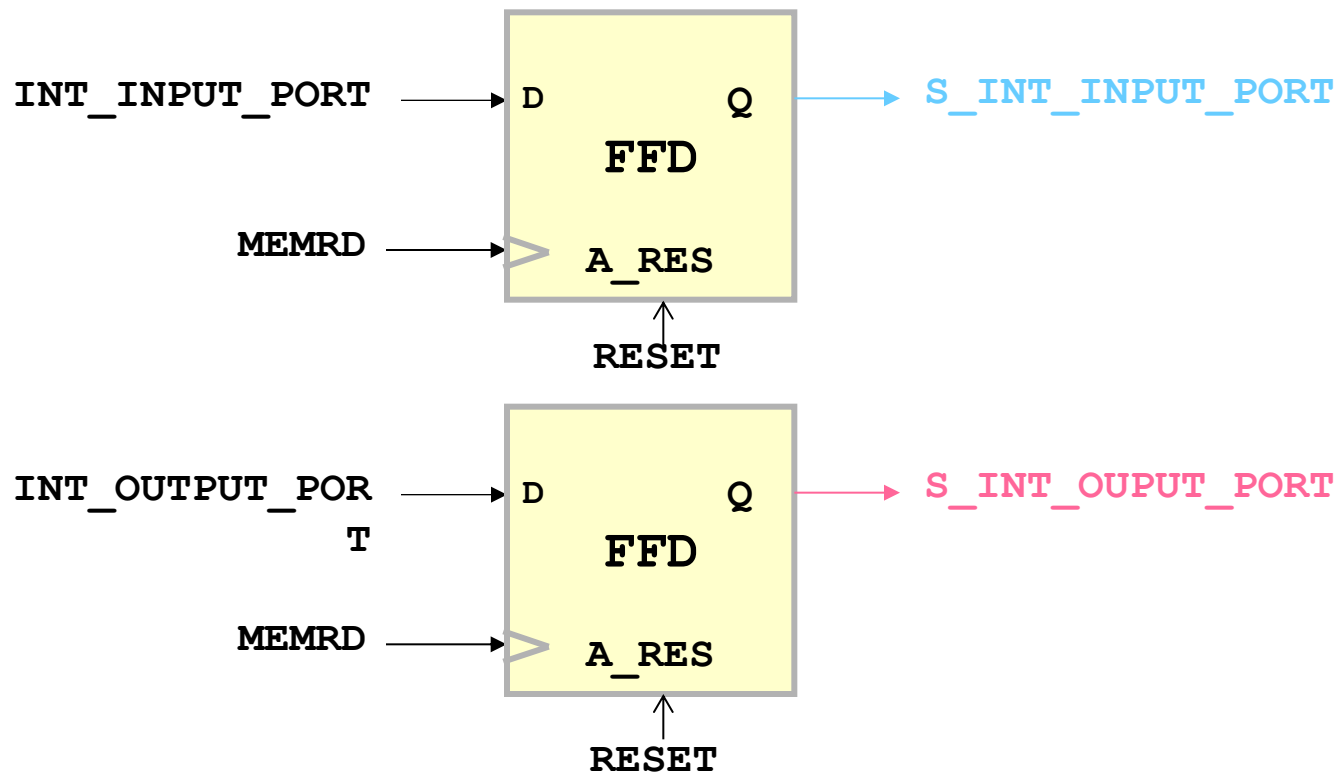
Si osservi che la richiesta di interruzione generata in base all'evento "numero di letture e scritture dispari" codificato dal segnale **INT_ODD_ODD**, è dipendente dalle letture e scritture nelle porte di input e output. Questo significa che, se in un determinato istante, il segnale **INT_ODD_ODD** è zero, potrà eventualmente divenire uno solo dopo lettura nella porta in input o una scrittura nella porta in output.

Inoltre, si ricorda che le richieste di interruzione dalle porte gestite con *handshake* possono avvenire in qualsiasi momento.

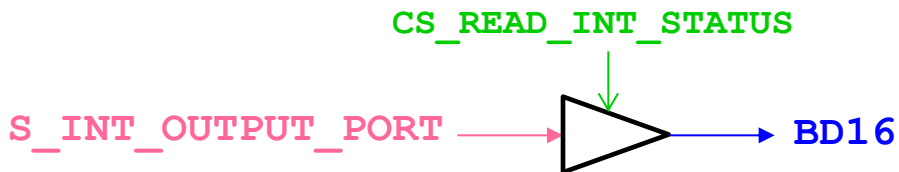
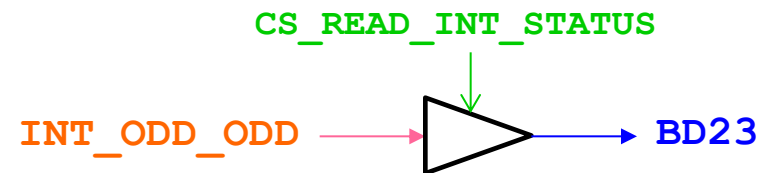
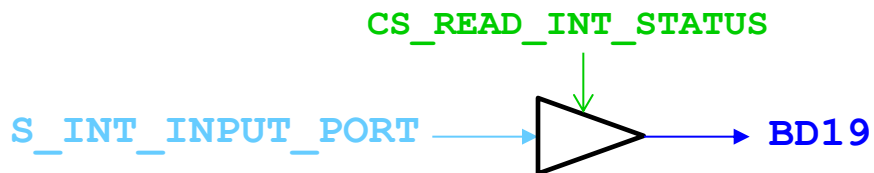
Per i motivi appena evidenziati, lo stato degli interrupt sarà letto dal DLX utilizzando la rete mostrata nella pagina seguente.

La sintesi del segnale **INT_ODD_ODD** sarà mostrata nelle pagine successiva mentre il segnale inviato sul pin INT del DLX risulterà:

$$\mathbf{INT_DLX} = \mathbf{INT_INPUT} + \mathbf{INT_OUTPUT} + \mathbf{INT_ODD_ODD}$$

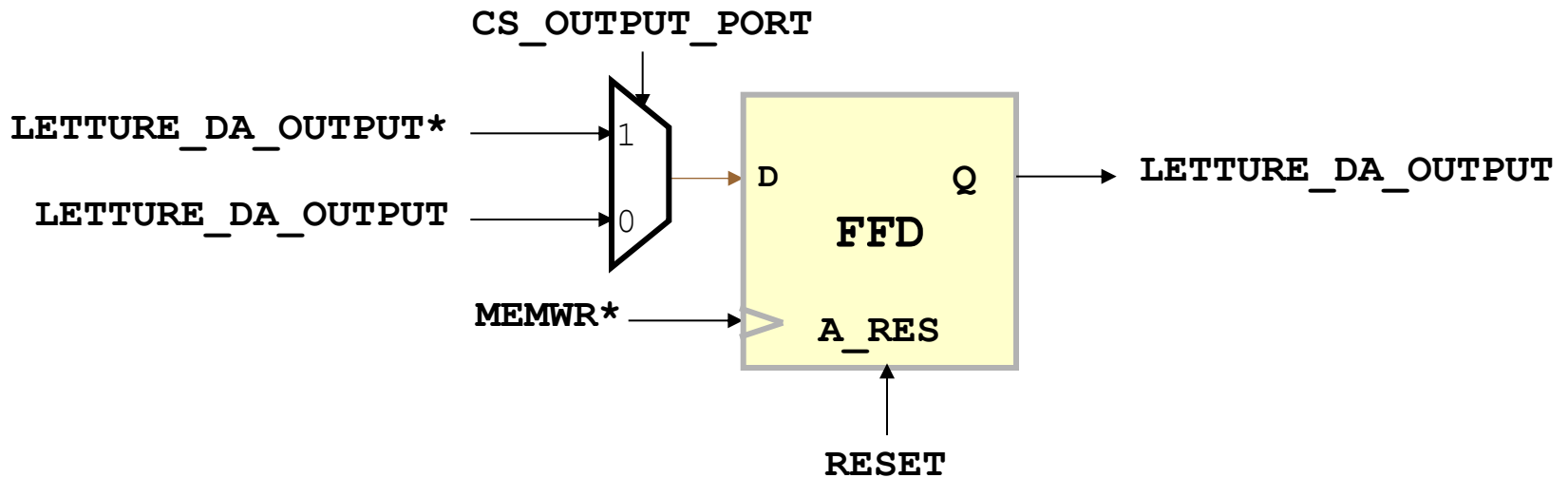
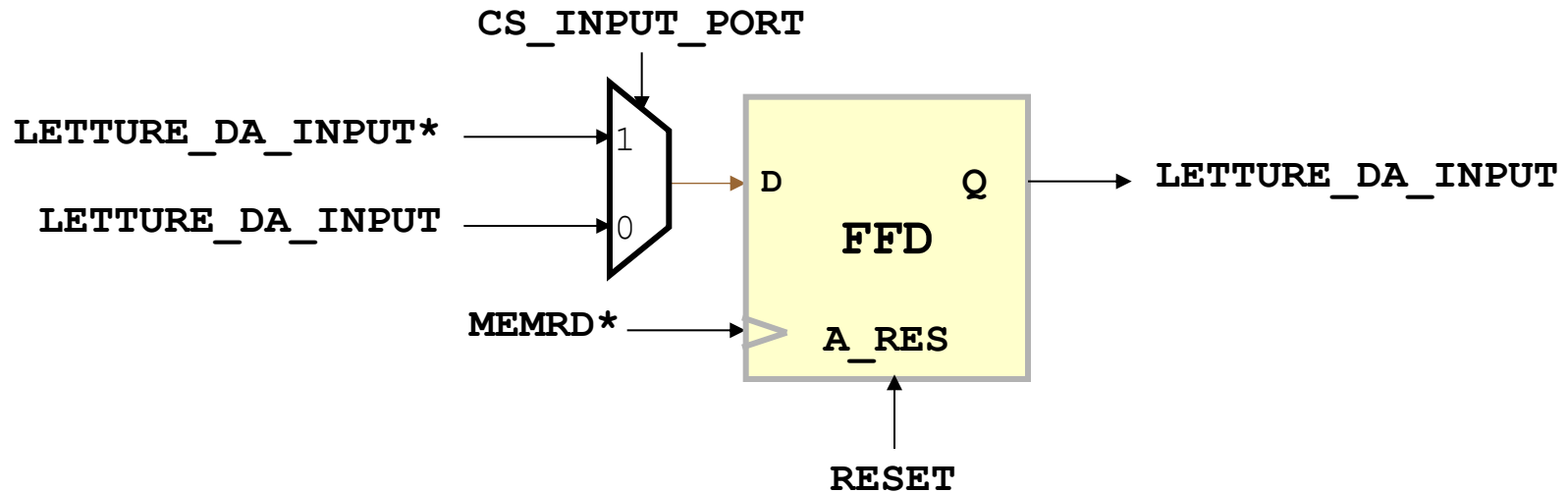


Lo stato delle richieste di interruzione, opportunamente campionato quando necessario in base a quanto descritto in precedenza, avviene mediante la rete seguente con una lettura (byte) a **CS_READ_INT_STATUS**.



OSSERVAZIONE: In realtà, non è necessario campionare e leggere lo stato di entrambe le porte di input e output. Infatti, come mostrato in seguito, è sufficiente farlo per una sola porta (ie, quella in input o quella in output).

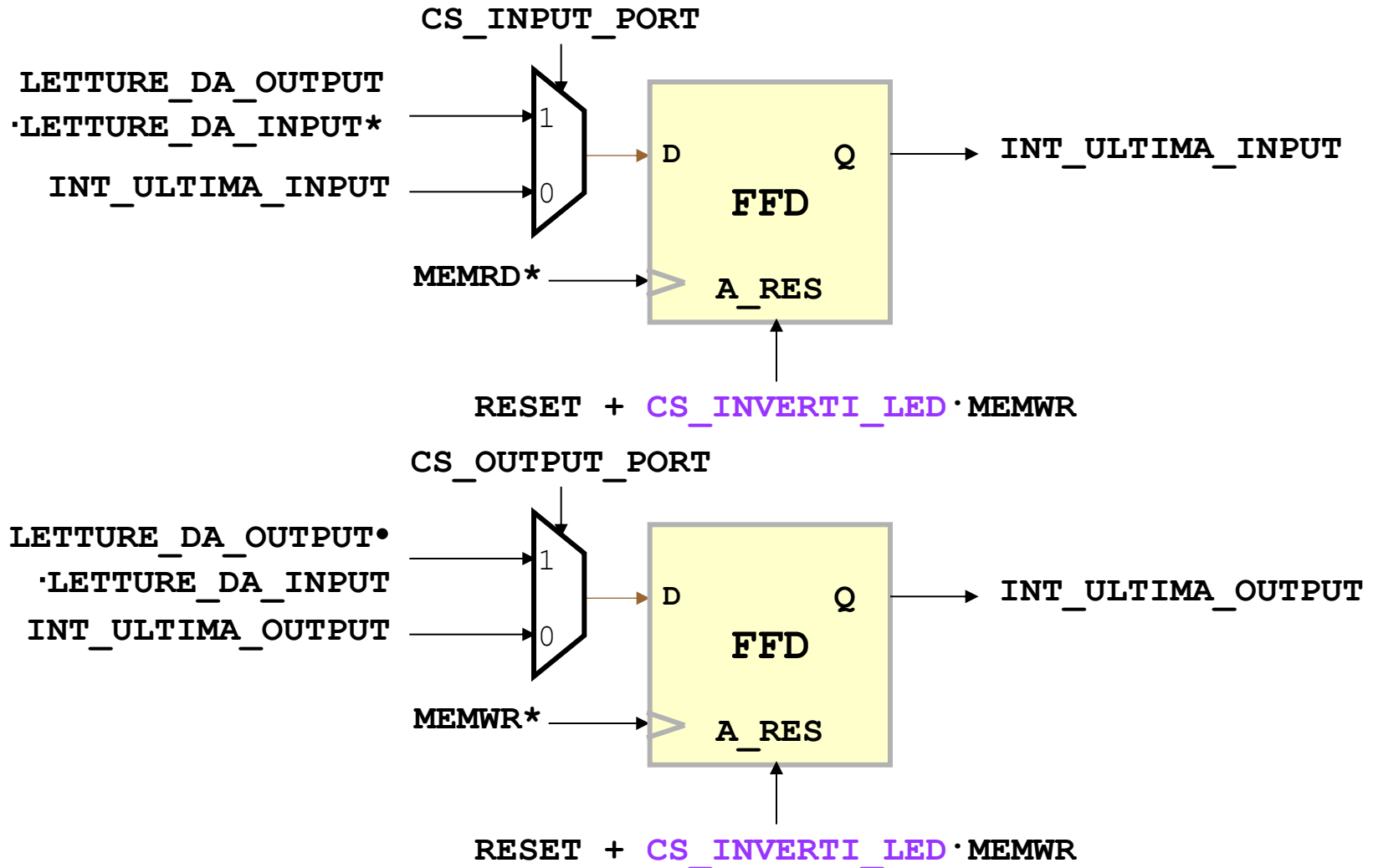
Mediante i due FFD seguenti è possibile tenere traccia del numero di accessi (pari o dispari) nella porta in input e output.



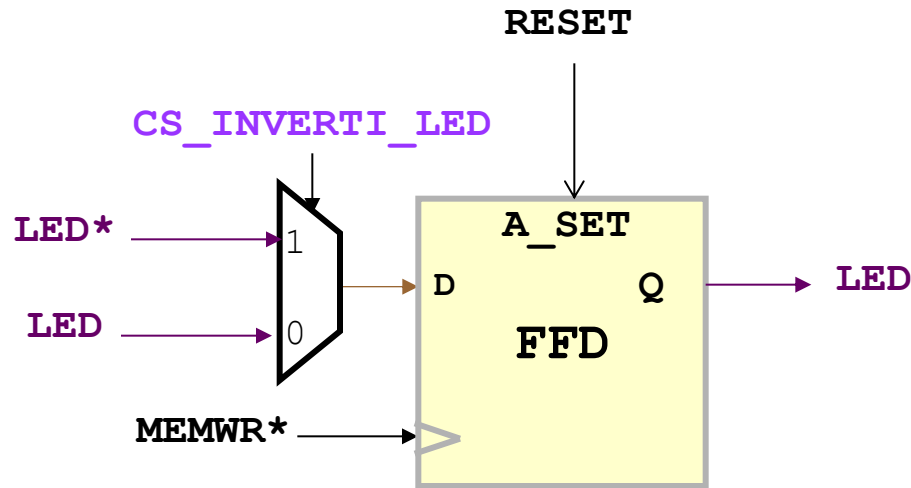
La rete seguente consente di generare il segnale **INT_ODD_ODD** come:

$$\text{INT_ODD_ODD} = \text{INT_ULTIMA_INPUT} + \text{INT_ULTIMA_OUTPUT}$$

Il segnale **INT_ODD_ODD** è portato al livello logico 0, mediante comandi asincroni **A_RES** dei due FFD, durante le scritture a **CS_INVERTI_LED**.



Infine, la rete seguente consente di gestire lo stato del LED (inizialmente accesso, come indicato nel testo del problema) mediante opportune scritture a **CS_INVERTI_LED** eseguite dall'interno dell'interrupt handler.



Nella pagina seguente è mostrato il codice dell'interrupt handler nel quale, per ogni esecuzione è gestita una sola interruzione. Inoltre, la priorità con la quale sono gestite le tre interruzione è la seguente:

- INT_ODD_ODD** (più prioritaria)
- INT_INPUT**
- INT_OUTPUT** (meno prioritaria)

L' interrupt handler risulta:

```
00000000: LHI R25,8000h           ; prepara indirizzo 80000000h in R25
00000004: LBU R26,2(R25)          ; legge a CS_READ_INT_STATUS
00000008: ANDI R27,R26,0080h      ; isola il bit 7 (maschera 100000002)
0000000C: BEQZ R27,IN_OR_OUT      ; se 0 gestisce input o output
00000010: SB R0,3(R25)            ; altrimenti inverte stato del LED
                                ; scrivendo a CS_INVERTI_LED
00000014: RFE                    ; ritorna dall' interrupt
```

: IN_OR_OUT

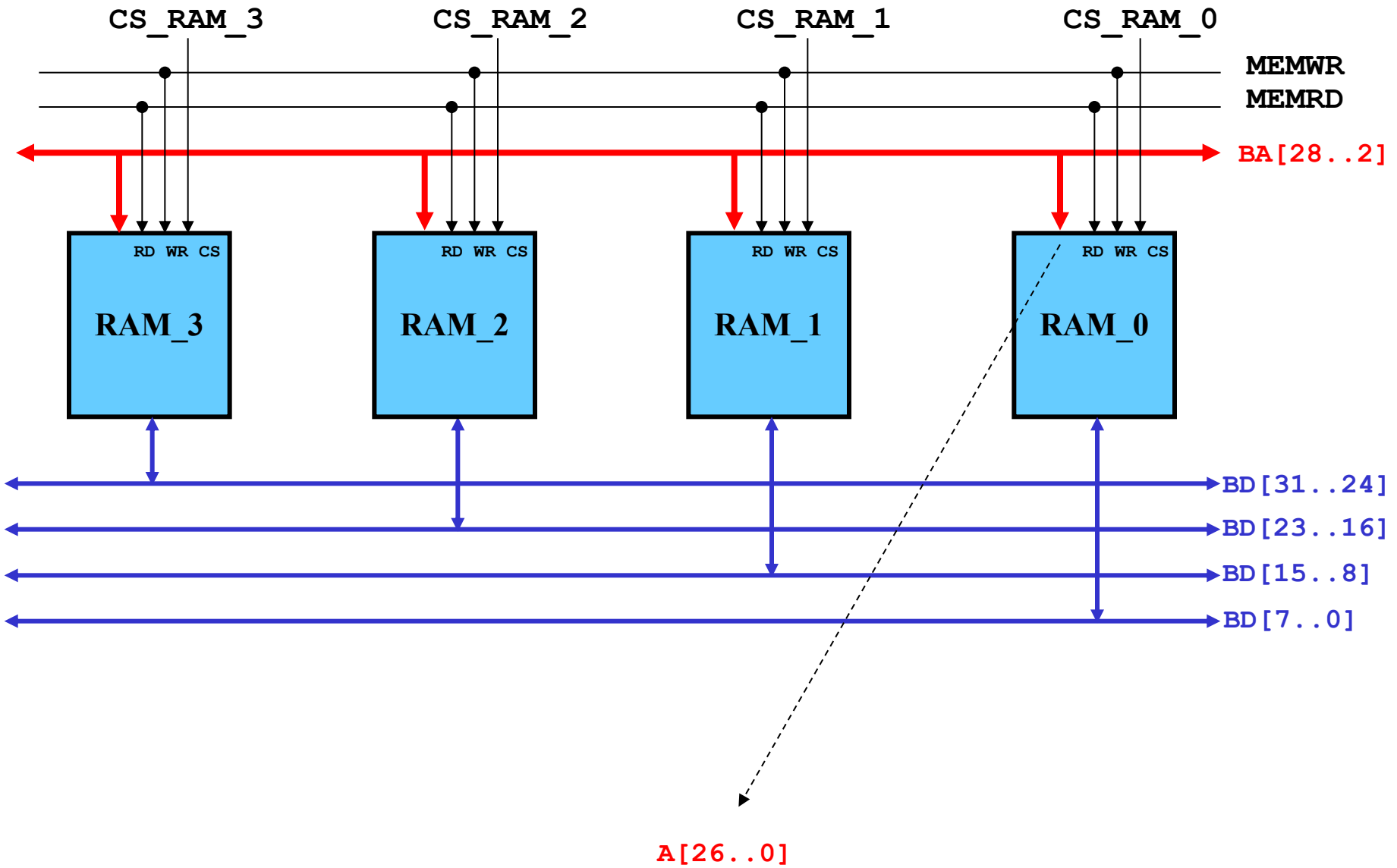
```
00000020: LHI R28,FFFFh           ; prepara indirizzo FFFF0000h in R28
00000024: ANDI R27,R26,0008h      ; isola il bit 3 (maschera 000010002)
00000028: BEQZ R27,OUTPUT         ; se 0 scrive su porta output
0000002C: LBU R27,0(R25)          ; altrimenti, legge da porta in input
00000030: SB R27,2000h(R28)        ; scrive a FFFF2000h
00000034: RFE                    ; ritorna dall' interrupt
```

: OUTPUT

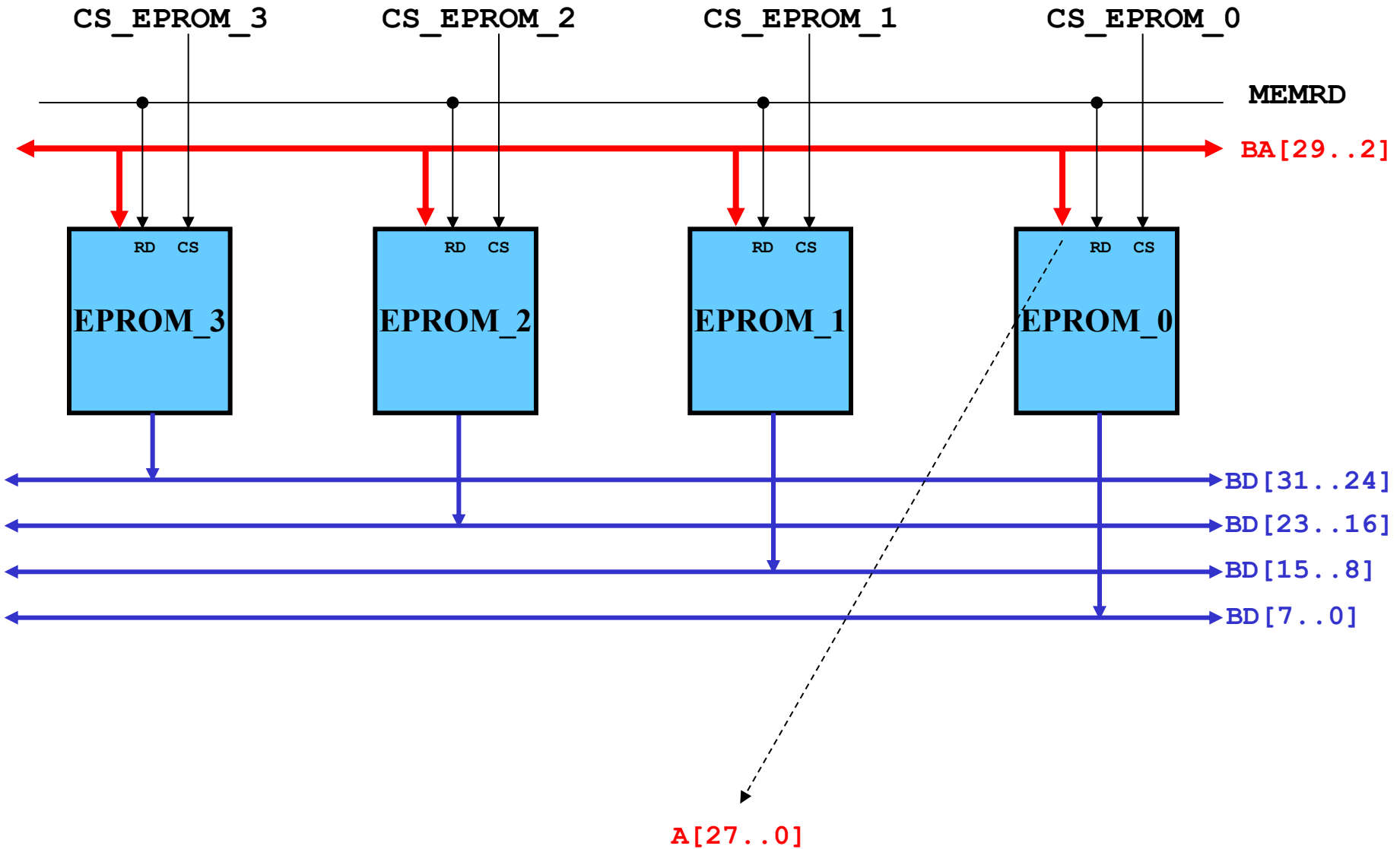
```
00000040: LBU R27,2004h(R28)      ; legge byte a FFFF2004h
00000044: SB R27,1(R25)           ; scrive su porta output
00000048: RFE                    ; ritorna dall' interrupt
```

Le costanti **IN_OR_OUT** e **OUTPUT** corrispondono, rispettivamente, a 10h e 10h.

Interfacciamento RAM



Interfacciamento EPROM



Esercizio 2

Rif. lucidi/lezioni.

Esercizio 3

Rif. lucidi/lezioni.